

Right now, Google is heavily subsidizing the computational cost of running these intense multi-agent workflows to get developers hooked on the Antigravity workflow. The moment you are locked into their specific system files, ~/.gemini/ configurations, and agent workflows, the friction to leave for Cursor or Windsurf becomes much higher.

The Developer Pushback

Unsurprisingly, developers don't love being locked into a single ecosystem. Because Google is keeping the gates closed, the community has resorted to making unofficial auth bypasses (like the opencode-antigravity-auth project on GitHub) just to extract the models out of Antigravity and use them in open-source CLIs where they actually have endpoint freedom.

So yes, your intuition is 100% correct. The lack of a "Custom API URL" box isn't a missing feature they forgot to code-it's a deliberate fence to keep you in the Google yard.

How Antigravity works - from prompt to execution

Context assembly (before I see your message)

Every turn the system builds a layered context:

- **System prompt** - Core identity, tool definitions, behavioural guidelines (web-dev aesthetics, planning-mode rules, artifact format, etc.)
 - **User rules** - AGENTS .md injected verbatim, highest priority
 - **KI summaries** - One-line summaries of all Knowledge Items so the model knows what has been learned across past sessions
 - **Workspace metadata** - Active file, cursor position, open documents, OS, current time
 - **Conversation history** - Full thread up to the token limit
- All of this is assembled before the model generates a single token.
-

Turn processing

When you send a message the model runs through four stages:

1. **Planning gate** - Decide whether a written plan is required (complex architectural changes) or the request can be executed directly (simple fixes).

2. **Research** - Read files, search the codebase, check KIs, or fetch URLs. This is where the loop-bug we just fixed lives.
3. **Execution** - Call tools (edit files, run commands, launch a browser sub-agent). Tools may be synchronous or async depending on dependencies.
4. **Verification** - Run npm run check, test suites, or take a browser screenshot to confirm the result.
5. **Response** - Return a concise summary, no fluff.

What the artifact/brain system adds

The brain/ directory gives the model persistent working memory across a long task:

- implementation_plan.md - Agreed design; re-read before each execution step so the model doesn't drift.
- task.md - Live checklist that is ticked off as work proceeds.
- walkthrough.md - Post-completion report containing diffs, screenshots, or recordings so you can see what changed without re-reading code.

Without this, a multi-session task would have to be reconstructed from conversation history alone, which degrades as the log grows.

1. Driving Vertex AI & Gemini Adoption

Google is playing catch-up to OpenAI and Anthropic in the developer mindshare space. By building a incredibly powerful "agent-first" IDE that handles the editor, terminal, and browser seamlessly, they want to prove that Gemini 3 and Vertex Model Garden are elite engineering tools. If they let you swap the core brain out for OpenRouter, you'd just use cheaper endpoints or rival models, and Google loses the telemetry and adoption data.

2. Under-the-Hood Heavy Lifting (The Agentic Control Loop)

Antigravity isn't just sending simple chat prompts. When you tell it to build a feature, it spins up an "Agent Manager" that runs deep, multi-step loops: planning, executing code, running terminal commands, and analyzing the UI via a Chrome extension.

* To do this reliably, Google tunes the system heavily around Gemini's native tool-calling, long context windows, and structural output capabilities.

* If they let users plug in random, unstable, or poorly quantized open-source models via external proxies, the complex agent loops would constantly break, making Antigravity look bad.

3. The Subsidized Lock-In

Other State

- `prompting/` - System prompt components (e.g., browser sub-agent instructions)
 - `browser_recordings/` - WebP video recordings from browser sub-agent sessions
 - `html_artifacts/` - Generated HTML artifact previews
 - `context_state/` - Saved mid-session context state
 - `conversations/` - Conversation index and metadata
 - `annotations/` - Persistent notes and annotations
 - `code_tracker/` - Tracks which files were modified per session
-

Document generated and formatted for readability.

You hit the nail right on the head. It is absolutely an intentional ecosystem play.

Google's official documentation for Antigravity explicitly states:

> "There is currently no support for bring-your-own-key or bring-your-own-endpoint."

Google didn't build a complete fork of VS Code and hand out free/subsidized access to massive frontier models out of pure altruism. They are executing a classic tech platform strategy, and there are a few big reasons why blocking OpenRouter or custom API base URLs serves their goals:

What the Knowledge Item (KI) system adds

KIs are the long-term memory layer. After a significant session, Antigravity distills what was learned (e.g., "*this repo uses X pattern for Y*") into a structured note stored under `knowledge/`. On future sessions, KI summaries appear automatically in the context, preventing duplicated research.

Practical advantages over a plain chat agent

Aspect Plain chat agent Antigravity Rules Re-stated each prompt or forgotten AGENTS.md injected every turn - hard priority Long-task tracking Relies on scrolling the conversation `task.md` + `implementation_plan.md` Cross-session memory None KIs persist learnings Verification Model decides when it's done Planning mode forces explicit verification step Workspace awareness None Active file, cursor, open docs injected automatically Browser interaction None Dedicated sub-agent with recording

The net effect is fewer "*I forgot what we agreed on*" errors on long tasks and consistent rule enforcement.

AGENTS.md

/Users/kai/Development/pi/AGENTS.md -
Hard rules injected every turn, highest
priority over all other instructions

App Data Root

File Location Purpose antigravity/ /Users/
kai/.gemini/antigravity/ Root - all Antigravity state
lives here mcp_config.json .../antigravity/
mcp_config.json MCP server configuration
browserAllolist.txt .../antigravity/
browserAllolist.txt Domains the browser sub-agent
may visit

Brain - Per-Conversation Working Memory

File Location Purpose brain/ / .../antigravity/
brain/ / One directory per session -
implementation_plan.md
implementation_plan.md Design doc written before

Knowledge Items - Cross-Session Long-Term Memory

File Location Purpose knowledge/ .../antigravity/
knowledge/ Persistent learnings distilled from past sessions
metadata.json knowledge//metadata.json KI
summary, timestamps, source conversation references
artifacts/ knowledge//artifacts/ Detailed files:
code snippets, architecture notes, patterns knowledge.lock
knowledge/knowledge.lock Prevents concurrent KI writes

complex changes; needs user approval Planning task.md
brain//task.md Live checklist tracking progress through the
plan Execution walkthroug.md brain//
walkthroug.md Post-completion summary with diffs,
screenshots, recordings After completion scratch/ brain//
scratch/ Throw-away scripts, temp data, debug output
Execution overview.txt
brain//.system_generated/logs/overview.txt
Raw JSONL transcript of every turn Auto-written